Purpose of Normalization (or) Schema Refinement

Normalization is the process of **organizing data in a database**.

- ✓ It **reduces repeated data (redundancy)**.
- ✓ It **removes anomalies** in a table.
- ✓ It **divides a large table into smaller tables**.

The main reason for normalization is **removing anomalies**.

Types of Anomalies

1. **Insertion Anomalycon**
2. **Deletion Anomaly**
3. **Update Anomaly**

Example: Unnormalized Table

STUDENT_DEPT

SID	SNAME	DEPTNO	DEPTNAME	HOD
101	Ravi	D1	IT	Kumar
102	Sita	D1	IT	Kumar
103	Ram	D2	ECE	Rao

Here **department information repeats for many students**.

Example: **IT department and HOD Kumar appear many times**.

1. Insertion Anomaly

Insertion anomaly occurs when **some data cannot be inserted without other data**.

Example:

We want to **insert a new department**.

SID	SNAME	DEPTNO	DEPTNAME	HOD
?	?	D4	CSE	Ramesh

But **there is no student record for this department**.

Since the table requires **student details**, we **cannot insert department information alone**.

This problem is called **Insertion Anomaly**.

2. Deletion Anomaly

Deletion anomaly occurs when **deleting a record also deletes other important information**.

Example

SID	SNAME	DEPTNO	DEPTNAME	HOD
101	Ravi	D1	IT	Kumar

If we **delete Ravi record**, then **CSE department information is also lost**.

This problem is called **Deletion Anomaly**.

3. Update Anomaly

Update anomaly occurs when **changing one value requires updating many rows**.

Example

SID	SNAME	DEPTNO	DEPTNAME	HOD
101	Ravi	D1	IT	Dr.Kumar
102	Sita	D1	IT	Kumar

Suppose the **HOD of CSE department changes from Kumar to Dr.Kumar**.

But **only one row is updated and another row is not updated**.

Now the **same department has two different HOD names**.

This creates **wrong and inconsistent data**.

This problem is called **Update Anomaly**.

Normalized Relations

To remove these anomalies, the table is divided into **two tables**.

STUDENT

SID	SNAME	DEPTNO
101	Ravi	D1
102	Sita	D1
103	Ram	D2

DEPARTMENT

DEPTNO	DEPTNAME	HOD
D1	CSE	Kumar
D2	ECE	Rao

This process is called **Normalization (Schema Refinement)**.

Keys in DBMS

A **key** is an attribute or a set of attributes that **uniquely identifies a tuple (row) in a relation (table)**.

Types of keys:

- ✓ **Super Key**
- ✓ **Candidate Key**
- ✓ **Primary Key**
- ✓ **Alternate Key**
- ✓ **Composite Key**
- ✓ **Surrogate Key**
- ✓ **Foreign Key**

Example : STUDENT

sid	**name**	**email**	**dept**
101	Ravi	ravi@gmail.com	CSE
102	Sita	sita@gmail.com	ECE
103	Ram	ram@gmail.com	IT

1. Super Key

A **Super Key** is a set of one or more attributes that **uniquely identifies each tuple in a table**.

- A Super Key **may contain extra attributes**.

Example:

Possible Super Keys in STUDENT table:

- ✓ {sid}
- ✓ {sid, name}
- ✓ {sid, email}
- ✓ {sid, name, dept}

All these uniquely identify rows, so they are **Super Keys**.

2. Candidate Key

A **Candidate Key** is a **minimal super key** — a Super Key **without unnecessary attributes**.

- ✓ It uniquely identifies tuples.

Example:

Candidate Keys in STUDENT table:

- ✓ sid
- ✓ email

Both uniquely identify rows and **cannot be reduced further**.

3. Primary Key

A **Primary Key** is the **candidate key selected to uniquely identify tuples in a table**.

- ✓ A table can have **only one primary key**.
- ✓ **Primary Key cannot be NULL** and **cannot have duplicate values**.

Example:

sid (PK)	name	email	dept
101	Ravi	ravi@gmail.com	CSE
102	Sita	sita@gmail.com	ECE
103	Ram	ram@gmail.com	IT

Here **sid** is the **Primary Key**.

4. Alternate Key

Alternate Keys are candidate keys **not selected as the primary key**.

Example:

Candidate keys:

- ✓ sid (chosen as Primary Key)
- ✓ email → becomes **Alternate Key**

5. Composite Key

A **Composite Key** is a key **formed using two or more attributes together** to uniquely identify a row.

Example Table: RESULT

student_id	subject	marks
101	DBMS	80
101	OS	75
102	DBMS	70

Here: **(student_id, subject)** together uniquely identify each row → **Composite Key**.

6. Surrogate Key

A **Surrogate Key** is an **artificially generated unique key used to identify each row**.

- ✓ Usually **a single column**
- ✓ **System generated** (like auto-increment)
- ✓ **Has no real-world meaning**
- ✓ Often used as a **Primary Key**

Example Table: STUDENT

SNO (Surrogate Key)	SID	Name
1	101	Ram
2	101	John
3	102	Sita
4	103	Hari

- ✓ **SNO** is auto-generated
- ✓ **SID may repeat**
- ✓ **SNO uniquely identifies each row**
- ✓ **SNO has no real-world meaning → Surrogate Key**

Note:

Prime Attributes and Non-Prime Attributes

1. Prime Attribute

A **Prime Attribute** is an attribute that **belongs to at least one Candidate Key**.

Candidate Keys:

- ✓ **sid**
- ✓ **email**

Prime Attributes:

- ✓ sid
- ✓ email

Because they are part of **candidate keys**.

2. Non-Prime Attribute

A **Non-Prime Attribute** is an attribute that **does not belong to any Candidate Key**.

Example

In the same table:

Non-Prime Attributes:

- ✓ name
- ✓ dept

Because they are **not part of any candidate key**.

Concept of functional dependency (FD)

Functional Dependency (FD)

A **functional dependency** is a relationship between attributes in a relation such that the value of one attribute (or a set of attributes) uniquely determines the value of another attribute.

It is represented as **FD**: **X → Y**.

- ✓ **X is called the Determinant**
- ✓ **Y is called the Dependent attribute**

From FD:

- ✓ **Attribute X determines attribute Y.**
- ✓ **If we know the value of X, we can uniquely determine the value of Y.**

Formal Definition

A functional dependency **X → Y** holds in relation **R** if for any two tuples **t1** and **t2** in **R**:

If
$\mathbf{t_1[X] = t_2[X]}$

Then
$\mathbf{t_1[Y] = t_2[Y]}$

This means that if two rows have the same value for X, they must also have the same value for Y.

Conditions for Functional Dependency

Rule 1: Unique Determinant (X is unique)

If the values of **X are unique**, then **X → Y** always holds because each value of **X** determines exactly one value of **Y**.

Example

sid	**name**
1	Ram
2	Ravi
3	Sita

FD: **sid → name**

In the above relation (table):

- ✓ **sid values are unique**
- ✓ Each **sid** determines exactly one **name**

Therefore, FD: **sid → name is valid**.

Rule 2: Repeating Determinant (X repeats)

If **X values repeat**, then:

- ✓ If **Y values are the same**, FD **is valid**.
- ✓ If **Y values are different**, FD **is invalid**.

Example

department	**hod**
IT	Kumar
IT	Kumar
ECE	Ramesh

FD: **department → hod**

In the above relation (table):

- ✓ **department IT repeats**
- ✓ But **hod is always Kumar**

Therefore, FD: **department → hod is valid**.

Example

department	**hod**
IT	Kumar
IT	Ramesh
ECE	Suresh

In the above relation (table):

- ✓ **department IT repeats**
- ✓ **hod values are different**

Therefore, FD: **department → hod is invalid**.

Types of Functional Dependencies in DBMS

1. Trivial Functional Dependency
2. Non-Trivial Functional Dependency
3. Fully Functional Dependency
4. Partial Functional Dependency
5. Transitive Functional Dependency
6. Multivalued Dependency

1. Trivial Functional Dependency

In Trivial Functional Dependency, **Y is a subset of X**.

If **X → Y** and **Y ⊆ X**, then it is called **Trivial Functional Dependency**.

Example

roll_no	**name**	**age**
42	abc	17

roll_no	name	age
43	pqr	18
44	xyz	18

Here

{roll_no, name} → name

is a **Trivial Functional Dependency**, because **name is part of {roll_no, name}**.

Similarly,

roll_no → roll_no

is also a **Trivial Functional Dependency**.

2. Non-Trivial Functional Dependency

In Non-Trivial Functional Dependency, **Y is not a subset of X**.

If **X → Y** and **Y is not a subset of X**, then it is called **Non-Trivial Functional Dependency**.

Example

roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18

Here

roll_no → name

is a **Non-Trivial Functional Dependency**, because **name is not a subset of roll_no**.

Similarly,

{roll_no, name} → age

is also a **Non-Trivial Functional Dependency**, because **age is not part of {roll_no, name}**.

3. Fully Functional Dependency

In Fully Functional Dependency, a **non-key attribute depends on the entire composite key**, not on part of the key.

Example

student_id	course_id	grade
101	DBMS	A
101	OS	B
102	DBMS	B

Here

(student_id, course_id) → grade

The attribute **grade** depends on **both student_id and course_id together**.

It does **not depend on student_id alone or course_id alone**.

So this is called **Fully Functional Dependency**.

4. Partial Functional Dependency

In Partial Functional Dependency, a **non-key attribute depends only on part of the composite key**, not on the whole key.

Example

student_id	**course_id**	**student_name**
101	DBMS	Ravi
101	OS	Ravi
102	DBMS	Sita

Here the composite key is **(student_id, course_id)**.

But

student_id → student_name

So **student_name depends only on student_id**, not on the whole key.

This is called **Partial Functional Dependency**.

5. Transitive Functional Dependency

In Transitive Functional Dependency, one attribute depends **indirectly on another attribute**.

If

A → B
B → C

then

A → C

This is called **Transitive Functional Dependency**.

Example

enrol_no	**name**	**dept**	**building_no**
42	abc	CSE	4
43	pqr	ECE	2
44	xyz	IT	1

Here

enrol_no → dept
dept → building_no

Therefore

enrol_no → building_no

This is an **indirect dependency**, so it is called **Transitive Functional Dependency**.

6. Multivalued Dependency

In Multivalued Dependency, **one attribute determines multiple independent values of another attribute**.

It is represented as **A →→ B**.

Example

roll_no	skill	hobby
101	Java	Cricket
101	Python	Cricket
101	Java	Music
101	Python	Music

Here

roll_no →→ skill
roll_no →→ hobby

A student can have **many skills and many hobbies**, and **skills are independent of hobbies**.

So this is called **Multivalued Dependency**.

Normal Forms based on Functional Dependency

1NF (First Normal Form):

A table is in **1NF** if:

- ✓ Each column contains **only one value**
- ✓ **No multiple values** in one cell
- ✓ Each record contains **atomic (single) values**

Example: Student Table

sid	name	mobileNo
1	Ram	9876543210, 9123456780
2	Ravi	9988776655

In the above table, the **mobileNo column contains two numbers** for **sid = 1**.

A column should **not contain multiple values**.

Therefore, this table is **NOT in 1NF**.

Decomposition into 1NF:

sid	name	mobileNo
1	Ram	9876543210
1	Ram	9123456780
2	Ravi	9988776655

- ✓ Each cell contains **only one value**
- ✓ No **multi-valued attributes**
- ✓ Now the table is in **1NF**

2NF (Second Normal Form):

A relation is in **2NF** if:

1. The relation must be in **1NF**
2. All **non-key (non-prime) attributes** are **fully functionally dependent on the Primary Key or Candidate Key**
3. There should be **no partial dependency**

Consider the Following Table

sid	courseId	studentName	courseName
1	C101	Ram	DBMS
1	C102	Ram	Java
2	C101	Ravi	DBMS

Candidate Key

(sid, courseId) → Candidate Key

Functional Dependencies (FDs)

1. sid → studentName
2. courseId → courseName
3. (sid, courseId) → all attributes

From FD's:

- ✓ **studentName** depends only on **sid**
- ✓ **courseName** depends only on **courseId**

These attributes depend on **part of the candidate key**, not the **whole candidate key**.

So **partial dependency exists**.

Therefore, the table is **NOT in 2NF**.

Decomposition into 2NF:

Student Table

FD: **sid → studentName**

sid	studentName
1	Ram
2	Ravi

Course Table

FD: **courseId → courseName**

courseId	courseName
C101	DBMS
C102	Java

Student_Course Table

FD: **(sid, courseId)**

sid	courseId
1	C101
1	C102
2	C101

- ✓ Now **no partial dependency**
- ✓ Every non-key attribute depends on the **whole candidate key**

Therefore, the relation is in **2NF**.

3NF (Third Normal Form):

A relation is in **3NF** if:

1. The relation must be in **2NF**
2. There should be **no transitive dependency**
3. **Non-key attributes should not depend on other non-key attributes**

Transitive Dependency
If **A → B** and **B → C**, then **A → C** is called transitive dependency.

Consider the Following Table

sid	studentName	deptId	deptName
1	Ram	D10	CSE
2	Ravi	D20	ECE
3	Sita	D10	CSE

Candidate Key

- ✓ **sid** → Candidate Key

Functional Dependencies (FDs)

1. **sid → studentName, deptId**
2. **deptId → deptName**

From FD's:

- ✓ **deptName** depends on **deptId**
- ✓ **deptId** depends on **sid**

So dependency is:

sid → deptId → deptName

This is **transitive dependency**.

Therefore, the table is **NOT in 3NF**.

Decomposition into 3NF

Student Table

FD: **sid → studentName, deptId**

sid	**studentName**	**deptId**
1	Ram	D10
2	Ravi	D20
3	Sita	D10

Department Table

FD: **deptId → deptName**

deptId	**deptName**
D10	CSE
D20	ECE

- ✓ Now **no transitive dependency**
- ✓ Non-key attributes depend only on the **candidate key**

Therefore, the relation is in **3NF**.

BCNF (Boyce–Codd Normal Form):

A relation is in **BCNF** if:

1. The relation must be in **3NF**
2. For every **Functional Dependency (X → Y), X must be a Candidate Key(super key)**
3. No **non-key determinant** is allowed

Every **determinant must be a candidate key**.

Consider the Following Table

student	course	instructor
Ram	DBMS	Kumar
Ravi	DBMS	Kumar
Ram	Java	Ramesh
Ravi	Java	Ramesh

Candidate Key

(student, course) → Candidate Key

Functional Dependencies (FDs)

1. **(student, course) → instructor**
2. **instructor → course**

From FD:

instructor → course

Here **instructor determines course**, but **instructor is not a candidate key**.

This **violates BCNF condition**.

Therefore, the table is **NOT in BCNF**.

Decomposition into BCNF:

The relation R(student, course, instructor) is decomposed into:

1. Instructor_Course(instructor, course)
2. Student_Instructor(student, instructor)

because FD: instructor → course violates BCNF.

Instructor_Course Table

FD: **instructor → course**

Instructor	Course
Kumar	DBMS
Ramesh	Java

Candidate Key → **instructor**

Student_Instructor Table

FD: **(student, instructor)** → candidate key

student	Instructor
Ram	Kumar

student	Instructor
Ravi	Kumar
Ram	Ramesh
Ravi	Ramesh

Candidate Key → **(student, instructor)**

- ✓ Now every **determinant is a candidate key**
- ✓ No BCNF violation

Therefore, the relation is in **BCNF**.

4NF(Fourth Normal Form):

A relation is in **4NF** if:

1. The relation must be in **BCNF**
2. There should be **no multi-valued dependency (MVD)**
3. One attribute should **not determine multiple independent attributes**

A table should **not contain two or more independent multi-valued attributes**.

Consider the Following Table

student	course	hobby
Ram	DBMS	Cricket
Ram	DBMS	Music
Ram	Java	Cricket
Ram	Java	Music

Candidate Key

(student, course, hobby) → Candidate Key

Multi-Valued Dependencies (MVD)

1. **student →→ course**
2. **student →→ hobby**

From FD's:

- ✓ A student can take **multiple courses**
- ✓ A student can have **multiple hobbies**

These two attributes are **independent of each other**.

From FD's:

- ✓ **course** depends on **student**
- ✓ **hobby** depends on **student**

But **course and hobby are independent**.

This creates **multi-valued dependency**.

Therefore, the table is **NOT in 4NF**.

Decomposition into 4NF

Student_Course Table

MVD: **student →→ course**

student	course
Ram	DBMS
Ram	Java

Candidate Key → **(student, course)**

Student_Hobby Table

MVD: **student →→ hobby**

student	hobby
Ram	Cricket
Ram	Music

Candidate Key → **(student, hobby)**

✓ Now **no multi-valued dependency problem**
✓ Independent attributes are stored in **separate tables**

Therefore, the relation is in **4NF**.

5NF (Fifth Normal Form):

A relation is in **5NF** if:

1. The relation must be in **4NF**
2. There should be **no join dependency**
3. The table cannot be further decomposed **without loss of information**

5NF is also called Projection-Join Normal Form (PJNF).

Consider the Following Table

student	subject	teacher
Ram	DBMS	Kumar
Ram	Java	Ramesh
Ravi	DBMS	Kumar

This table shows:

- Which **student studies which subject**

- Which **teacher teaches which subject**

Candidate Key

(student, subject, teacher)

Join Dependency

The table can be decomposed into **three smaller tables**.

Decomposition into 5NF

Student_Subject Table

student	**subject**
Ram	DBMS
Ram	Java
Ravi	DBMS

Candidate Key → **(student, subject)**

Subject_Teacher Table

subject	**teacher**
DBMS	Kumar
Java	Ramesh

Candidate Key → **(subject, teacher)**

Student_Teacher Table

student	**teacher**
Ram	Kumar
Ram	Ramesh
Ravi	Kumar

Candidate Key → **(student, teacher)**

If we **join these three tables**, we can **reconstruct the original table**.

- ✓ No data loss
- ✓ No redundancy

Therefore the relation is in **5NF**.

Lossless join and dependency preserving decomposition

1. Lossless Join Decomposition

Lossless join decomposition means decomposing a relation into smaller relations such that when they are joined again, the **original relation is obtained without losing any tuple or generating extra tuples**.

Example

Relation: **STUDENT(sid, sname, instructor)**

Functional Dependencies:

1. sid → sname
2. sid → instructor

Decomposition:

- ✓ R1(sid, sname)
- ✓ R2(sid, instructor)

Common attribute = sid
sid is a candidate key in R1.

Joining R1 and R2 gives original STUDENT relation.

Hence decomposition is lossless.

2. Dependency Preserving Decomposition

Dependency preserving decomposition means all functional dependencies of the original relation can be checked without performing join operation on decomposed relations.

Example

Relation: **R(A, B, C)**

Functional Dependencies:

1. A → B
2. B → C

Decomposition:

- ✓ R1(A, B) → preserves A → B
- ✓ R2(B, C) → preserves B → C

All dependencies are preserved.

Attribute Closure and Candidate Key

Attribute Closure (X^+)

The **closure of an attribute set X (X^+)** is the set of all attributes that can be determined from X using functional dependencies.

Example:

Relation: R(A, B, C, D)

FDs:

A → B
B → C
C → D

Find A^+

Start:

$A^+ = \{A\}$

Apply FD rules:

$A \rightarrow B \Rightarrow$ add $B \rightarrow \{A, B\}$
$B \rightarrow C \Rightarrow$ add $C \rightarrow \{A, B, C\}$
$C \rightarrow D \Rightarrow$ add $D \rightarrow \{A, B, C, D\}$

$A^+ = \{A, B, C, D\}$

A^+ contains all attributes → **A is a super key**

If A is minimal → **A is also a candidate key**

Candidate Key

A **Candidate Key** is a **minimal set of attributes** whose closure contains all attributes of the relation.

How to Find Candidate Key

1) R (A, B, C, D, E)

2) FD's:

$A \rightarrow B$
$B \rightarrow C$
$DE \rightarrow A$

3) Start with attributes that are not dependent (not appearing on RHS).

RHS = {B, C, A}

Missing attributes = {D, E}

Start candidate key = {D, E}

4) Find closure.

$\{D, E\}^+ = \{A, B, C, D, E\}$

Closure contains all attributes → Candidate Key = {D, E}